



SAPIENZA  
UNIVERSITÀ DI ROMA

# n-Queens: $A^*$ vs CSP

## ARTIFICIAL INTELLIGENCE

**Professors:**

Marco Favorito  
Francesco Fuggitti

**Student:**

Chiara Mecchia  
(2205187)

---

Academic Year 2025/2026

# Contents

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                      | <b>2</b>  |
| <b>2</b> | <b>Problem Modeling</b>                  | <b>3</b>  |
| 2.1      | State Representation . . . . .           | 3         |
| 2.2      | Goal Test . . . . .                      | 3         |
| <b>3</b> | <b>Implementation of A*</b>              | <b>4</b>  |
| 3.1      | Core Logic and Data Structures . . . . . | 4         |
| 3.2      | Heuristic Function . . . . .             | 5         |
| <b>4</b> | <b>Implementation of CSP Reduction</b>   | <b>6</b>  |
| 4.1      | Variables and Domains . . . . .          | 6         |
| 4.2      | Constraints . . . . .                    | 6         |
| 4.3      | Solver Integration and Logic . . . . .   | 7         |
| <b>5</b> | <b>Experimental Results</b>              | <b>8</b>  |
| 5.1      | Experiment Setup . . . . .               | 8         |
| 5.2      | Results Visualization . . . . .          | 8         |
| 5.3      | Analysis . . . . .                       | 9         |
| <b>6</b> | <b>Conclusions</b>                       | <b>10</b> |

# Chapter 1

## Introduction

This project explores the implementation and performance evaluation of two fundamental Artificial Intelligence techniques:

- **Heuristic Search (A\*)**
- **Constraint Satisfaction Problems (CSP)**

The chosen problem for this experiment is the classic ***n*-Queens puzzle**: the objective is to place  $n$  non-attacking queens on an  $n \times n$  chessboard. This problem was selected because it offers a clear scaling parameter ( $n$ ), allowing for a rigorous analysis of how different algorithms handle increasing complexity.

# Chapter 2

## Problem Modeling

### 2.1 State Representation

A critical aspect of solving the  $n$ -Queens problem is the efficiency of the state representation. Instead of using a full  $n \times n$  matrix, which would lead to a massive state space ( $2^{n^2}$ ), the state was modeled as a **tuple of integers**.

- **State:** A tuple  $S = (r_0, r_1, \dots, r_k)$ , where  $r_i$  is the row index of the queen in column  $i$ .
- **Implicit Constraint:** By using this representation, the constraint that no two queens can be in the same column was implicitly satisfied.
- **Transition Model:** To move from a state of length  $k$  to  $k + 1$ , the attempt to place a queen in the next column  $k + 1$  at every row  $j \in \{0, \dots, n - 1\}$  that does not violate row or diagonal constraints was implemented.

### 2.2 Goal Test

The goal is reached when the length of the tuple equals  $n$ , indicating that  $n$  queens have been safely placed on the board.

# Chapter 3

## Implementation of A\*

### 3.1 Core Logic and Data Structures

The A\* algorithm was implemented using a **Priority Queue** (via Python's `heapq` library) to manage the frontier. The implementation adheres to the following search policies:

1. **Duplicate Elimination:** A `closed_set` stores every expanded state. Before expanding a node, the algorithm checks if its state is already in this set, in order to prevent redundant computations and cycles, ensuring the search remains efficient.
2. **No Reopening:** Once a state is moved to the `closed_set`, it is never re-added to the frontier.

#### Optimality and State Space Structure

Since the problem was modeled as an incremental construction (placing queens column by column), each valid configuration can be reached through a **unique sequence of actions**. This means that the search tree does not contain multiple paths to the same state with different costs  $g(n)$ . Consequently, the first time a state is discovered, it is guaranteed to be via its only path. This property makes the "No Reopening" policy perfectly suitable for this specific problem without sacrificing the optimality of the solution.

#### Frontier Management

To handle the open list efficiently, a `frontier_tracker` was used to map states to their best  $f$ -value. This allows the algorithm to check if a newly generated node is already in the frontier with a higher cost. If so, the node is updated in the priority queue.

## 3.2 Heuristic Function

The chosen heuristic was the **Remaining Queens** heuristic, defined as:

$$h(S) = n - \text{length}(S) \quad (3.1)$$

This function estimates the cost to reach the goal by calculating how many queens are yet to be placed on the board.

There are two properties that guarantee that the A\* search will always return an optimal solution:

- **Admissibility:** Since each placement has a uniform cost of 1 and exactly  $n - \text{length}(S)$  more queens have to be placed to reach a goal state,  $h(S)$  corresponds exactly to the true cost  $h^*(S)$ . Therefore,  $h(S) \leq h^*(S)$ , satisfying the admissibility condition.
- **Consistency:** For any state  $S$  and its successor  $S'$ , the cost of the transition is  $c = 1$ . Since  $h(S) = n - \text{length}(S)$  and  $h(S') = n - (\text{length}(S) + 1)$ , we have  $h(S) = 1 + h(S')$ . This satisfies the consistency inequality  $h(S) \leq c(S, a, S') + h(S')$ , ensuring that the total evaluation function  $f(n)$  is non-decreasing along any path.

# Chapter 4

## Implementation of CSP Reduction

The second technique involves reducing the  $n$ -Queens problem to a **Constraint Satisfaction Problem (CSP)**. This approach shifts the perspective from finding a path in a state space to finding a valid assignment for a set of variables that satisfies all global and local constraints.

### 4.1 Variables and Domains

To model the problem efficiently, each column of the chessboard was mapped to a specific variable:

- **Variables:**  $X = \{C_0, C_1, \dots, C_{n-1}\}$ , where each  $C_i$  represents the queen's position in the  $i$ -th column.
- **Domains:** For each variable  $C_i$ , the domain is  $D_i = \{0, 1, \dots, n-1\}$ , representing the available row indices.

This setup ensures by design that no two queens can occupy the same column, reducing the number of explicit constraints to be handled by the solver.

### 4.2 Constraints

The problem logic is enforced through two primary types of constraints:

#### 1. Row Constraint

To ensure that no two queens share the same row, we apply a **Global Constraint**:

- **AllDifferentConstraint**( $C_0, \dots, C_{n-1}$ ): This is more efficient than defining  $n(n-1)/2$  binary constraints ( $C_i \neq C_j$ ), as it allows the solver to perform more aggressive pruning across all variables simultaneously.

## 2. Diagonal Constraint

Diagonal conflicts occur if the distance between the rows of two queens is equal to the distance between their columns. This can be implemented as a set of binary constraints for every pair of columns  $(i, j)$  with  $i < j$ :

$$|C_i - C_j| \neq |i - j| \quad (4.1)$$

In the code, this is handled by a lambda function that checks the absolute difference between values  $(r_1, r_2)$  against the pre-calculated distance between columns  $(c_1, c_2)$ .

## 4.3 Solver Integration and Logic

The implementation **automatically generates** the CSP model based on the input size  $n$ . The process involves:

1. Dynamically adding variables and their respective domains to the solver instance.
2. Iteratively applying the diagonal check for every possible pair of variables.
3. Invoking the solver's search engine, which utilizes *backtracking* combined with *constraint propagation* techniques.

Unlike A\*, the CSP solver is **proactive**: through constraint propagation, it can prune values from the domains of unassigned variables as soon as a partial assignment is made, effectively "predicting" and avoiding future conflicts before they occur.



# Chapter 5

## Experimental Results

### 5.1 Experiment Setup

The experimental evaluation was conducted on a range of problem sizes  $n \in \{4, 6, 8, 10, 12, 13, 14\}$ . For each instance, both the custom A\* implementation and the CSP solver integration were executed. The metrics collected include:

- **Execution Time:** Measured in seconds using a high-resolution timer.
- **Nodes Expanded:** The total number of states removed from the frontier and processed.
- **A\* Memory Peak:** The maximum total number of nodes stored simultaneously in both the frontier (Open List) and the explored set (Closed Set).

### 5.2 Results Visualization

The following table summarizes the performance metrics observed during the experiments:

| N  | A* Time (s) | CSP Time (s) | A* Nodes Exp. | A* Mem Peak |
|----|-------------|--------------|---------------|-------------|
| 4  | 0.0000      | 0.0001       | 11            | 14          |
| 6  | 0.0002      | 0.0003       | 35            | 47          |
| 8  | 0.0005      | 0.0009       | 86            | 123         |
| 10 | 0.0008      | 0.0007       | 127           | 211         |
| 12 | 0.0008      | 0.0019       | 91            | 268         |
| 13 | 0.0027      | 0.0038       | 313           | 520         |
| 14 | 0.0133      | 0.0030       | 1456          | 1697        |

Table 5.1: Experimental metrics comparing A\* Search and CSP Solver performance

The metrics were plotted to visualize the scaling behavior of both algorithms.

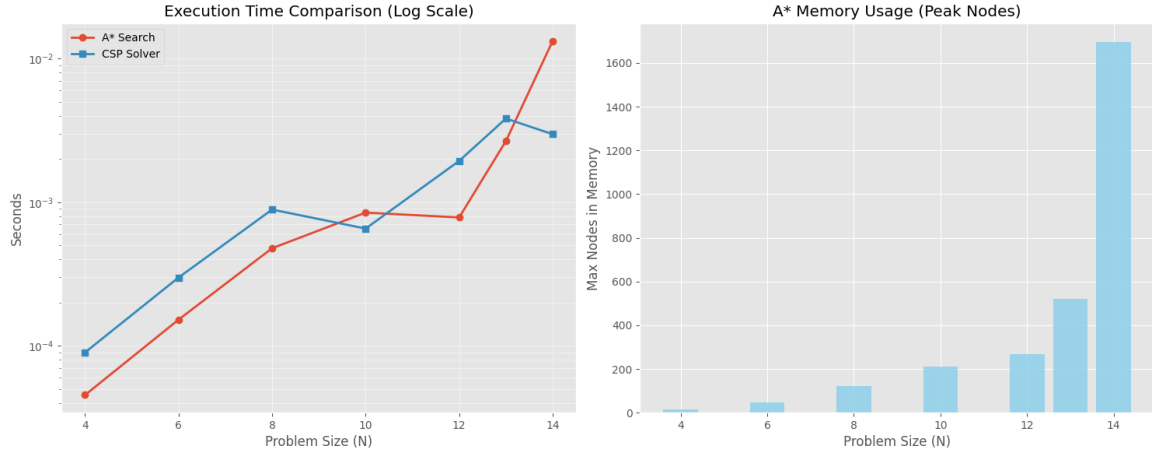


Figure 5.1: Execution time (log scale) and A\* memory peak across problem sizes.

## 5.3 Analysis

### Non-monotonic Complexity

At  $N = 12$  the A\* algorithm expands only 91 nodes, which is significantly lower than the 127 nodes expanded for  $N = 10$ . This highlights that the difficulty of the  $n$ -Queens problem does not scale in a strictly monotonic way with  $N$ .

### Execution Time and Scaling

Up to  $N = 13$ , both algorithms perform in the millisecond range. However, at  $N = 14$ , we observe a sharp increase in the A\* execution time (0.0133s), while the CSP solver remains extremely stable (0.0030s). This demonstrates the effectiveness of **Constraint Propagation**: while A\* must explore and evaluate nodes, the CSP solver proactively prunes the search space. At  $N = 14$ , the benefits of this pruning outweigh the overhead of maintaining the constraint graph, making the CSP solver more efficient as the problem scales.

### Memory Bottleneck

The **Mem Peak** for A\* shows a consistent exponential growth, reaching 1697 nodes for  $N = 14$ . In a search-based approach, memory is often the primary bottleneck because the algorithm must store the `closed_set` to ensure duplicate elimination.

# Chapter 6

## Conclusions

While  $A^*$  is a powerful tool for pathfinding and general search, the  $n$ -Queens problem highlights its limitations when compared to specialized CSP solvers. The implementation of "No Reopening" and "Duplicate Elimination" was fundamental to keep the  $A^*$  search feasible for  $n \leq 14$ .